

Member Management and Org Hub

Last updated: 2026-05-11

ArtHound organisations (studios and vendors) are multi-user. The member management system controls who belongs to an org, at what privilege level, and how new users join. The org hub is the in-app surface for all of this.

Concepts

Org: a studio or vendor record. Every user belongs to exactly one org at a time.

Member role: three tiers per org: - `owner`, full control; can transfer ownership, promote/demote admins, remove any member - `admin`, can approve/decline join requests, manage `user`-role members, regenerate the invite code - `user`, read access to the org hub; cannot take management actions

Every org has exactly one owner at all times.

Invite code: an 8-character alphanumeric code (A-Z, 0-9) stored on the org record. Sharing this code allows new users to request membership. Codes are unique globally. Admins can regenerate the code at any time; the old code immediately stops working.

Join request: created when a user signs up with a valid invite code. The user's account exists but they have no membership row until an admin approves. While pending, the user sees a holding screen and cannot access the product.

Membership cache: the auth layer (`lib/auth.py`) caches each user's resolved role and membership for 60 seconds. Changes (approval, role change, removal) are reflected within that window.

Signup Flow

There are two paths through the signup modal.

Path 1: Create a new org (no invite code)

1. User selects org type (studio or vendor), provides org name, email, and password
2. Backend (`POST /api/auth/signup`): creates the Supabase auth user, creates the org row, inserts a `studio_members` / `vendor_members` row with `member_role = owner`, stamps `invite_code` on the org
3. User is immediately active, no approval step

Path 2: Join an existing org (invite code)

1. User obtains an invite code from an existing org member
2. Enters the code in the signup modal; `GET /api/invite-code/{code}/resolve` previews the org name and type before committing
3. Provides email and password
4. Backend: creates the auth user, inserts a `studio_join_requests` / `vendor_join_requests` row (`status = pending`), **no membership row yet**
5. User lands on the `PendingApproval` page and cannot proceed
6. An org admin approves the request via the org hub → membership row inserted as `user` role
7. Within 60 seconds (cache TTL), the user's next page load resolves their membership and they gain access

The resolve endpoint is rate-limited to 10 requests/minute/IP to prevent org enumeration.

Join Request Lifecycle

```
studio_join_requests / vendor_join_requests
status: pending → accepted | declined
```

Status	Meaning
<code>pending</code>	Awaiting admin action; unique partial index enforces one pending request per user per org
<code>accepted</code>	Admin approved; membership row exists
<code>declined</code>	Admin rejected; the partial index allows re-submission (a declined user can request again)

Accepting a request (`POST /api/org/join-requests/{id}/accept`) atomically: 1. Inserts the membership row (`member_role = user`) 2. Sets `status = accepted` , `resolved_at` , `resolved_by` on the request row 3. Invalidates the membership cache for that user

Declining (`POST /api/org/join-requests/{id}/decline`) sets status to `declined` without creating a membership row.

Org Hub

`GET /api/org/hub` returns everything the hub page needs in one call:

```

{
  "org": {
    "id": " ... ",
    "name": "Studio A",
    "handle": "@pixel-forge", // vendor only
    "invite_code": "ABC12345",
    "initialized_at": "2026-05-01T ... "
  },
  "members": [
    { "user_id": " ... ", "email": "alice@ ... ", "member_role": "owner", "joined_at": " ... ", "..."
    ...
  ],
  "pending_requests": [ // admin/owner only
    { "id": " ... ", "user_id": " ... ", "email": "bob@ ... ", "created_at": " ... " },
    ...
  ]
}

```

The hub UI (`frontend/src/pages/OrgHub.jsx`) surfaces:

- Org name, handle (vendor), and member count
- Invite code with a one-click copy button
- Admin action to regenerate the invite code (`POST /api/org/invite-code/regenerate`)
- Members list sorted by role (owner → admin → user), with inline role management for admins/owners
- Pending join requests with approve/decline actions (admin/owner only)

Role Management

Change a member's role

`PATCH /api/org/members/{user_id}/role` , body: `{role: "owner" | "admin" | "user"}`

Permission rules: - Admin or owner can promote/demote between `user` and `admin` - Only the owner can promote someone to `owner` (this triggers an ownership transfer, see below) - No one can directly demote the current owner; ownership must be transferred first

Remove a member

`DELETE /api/org/members/{user_id}`

- Admin or owner can remove `user` -role members
- Only the owner can remove `admin` -role members
- The org owner cannot be removed (ownership must be transferred first)

Ownership transfer

When promoting a user to `owner` , the backend calls the `transfer_org_ownership(org_type, org_id, current_owner_id, new_owner_id)` Postgres RPC. This runs as a single transaction: the new user

becomes `owner`, the current owner becomes `admin`. There is no gap where the org has zero owners.

Database Schema

`studio_members` / `vendor_members`

```
(studio_id | vendor_id, user_id) PK
member_role: owner | admin | user
created_at
```

`studio_join_requests` / `vendor_join_requests`

```
id (uuid PK)
org_id (studio_id or vendor_id)
user_id
status: pending | accepted | declined
created_at, resolved_at, resolved_by
```

Partial unique index on `(org_id, user_id) WHERE status = 'pending'`, one live request per user per org.

API Reference

All endpoints require JWT auth unless noted. Studio and vendor routes are symmetric.

Method	Path	Auth	Description
GET	/api/invite-code/{code}/resolve	none (rate-limited)	Preview org from invite code
POST	/api/auth/signup	none	Create account (both paths)
GET	/api/org/hub	member	Org summary, members, pending requests
GET	/api/org/join-requests	admin/owner	List pending join requests
POST	/api/org/join-requests/{id}/accept	admin/owner	Approve request → create membership
POST	/api/org/join-requests/{id}/decline	admin/owner	Decline request
POST	/api/org/invite-code/regenerate	admin/owner	Issue new invite code
PATCH	/api/org/members/{user_id}/role	admin/owner	Change member role
DELETE	/api/org/members/{user_id}	admin/owner	Remove member
GET	/api/user/me	member or pending	Current user profile and membership state

Known Gaps

No email notification on join request: when a user requests membership, admins receive no notification. They must check the org hub manually. An email or in-app notification on new join requests is a planned improvement (see notification system TODO).

No self-service role request: users cannot request a role upgrade; only admins/owners can change roles.

60-second cache lag: removed or demoted members retain their access level for up to 60 seconds. This is intentional for performance but means revocation is not instantaneous.